the grid, so we wrap around to the beginning. This makes the grid into a circular grid.

Each element in the grid (either 1 or 0) represents an electron being spin-up or spin-down. In reinforcement learning jargon, the electrons are individual *agents* in the environment. Agents need to have value functions and policies, so they cannot merely be a binary number. The binary number on the grid represents the action of the agent, choosing to be either spin-up or spin-down. Hence, we need to model our agents using a neural network. We will use a Q-learning approach rather than the policy gradient method. In listing 9.4 we define a function that will create parameter vectors to be used in a neural network.

Listing 9.4. The 1D Ising model: Generate neural network parameters

```
def gen_params(N,size):          1
    ret = []
    for i in range(N):
        vec = torch.randn(size) / 10.
        vec.requires_grad = True
        ret.append(vec)
    return ret
```

- ***1*** This function generates a list of parameter vectors for a neural network.

Since we will be using a neural network to model the Q function, we need to generate the parameters for it. In our case, we will use a separate neural network for each agent, although this is unnecessary; each agent has the same policy, so we could re-use the same neural network. We'll do this just to show how it works; for the later examples we will use a shared Q function for agents with identical policies.

Since the 1D Ising model is so simple, we will write the neural network manually by specifying all the matrix multiplications rather than using PyTorch's built-in layers. We need to make a Q function that accepts a state vector and a parameter vector, and in the function body we unpack the parameter vector into multiple matrices that form each layer of the network.

Listing 9.5. The 1D Ising model: Defining the Q function